LECTURE 17

Introduction to Verified Functional Synthesis

We are now ready to move beyond inductive synthesis to richer forms of specification. Within these richer forms of specification, the literature distinguishes between *functional synthesis*, where the goal is to synthesize functions that map inputs to outputs, or more generally an input state to an output state, and *reactive synthesis*, where the goal is to synthesize a system that will run continuously for an unbounded amount of time and will react to an ongoing stream of inputs with a sequence of outputs. Reactive synthesis is itself a broad field that is unfortunately beyond the scope of this course. The focus for this unit will be on functional synthesis.

The general synthesis problem is to find a program P that satisfies a specification. There are three major issues in this form of synthesis: The first is to establish the form of the specification. The second is the approach to be used to establish the correctness of the eventual solution. And finally, there is the search strategy. All three of these issues are tightly intertwined. Different forms of specification can enable different verification strategies, and the interplay between the verification strategy and the search is crucial in enabling scalability to complex problems. At one extreme of this interplay are correct-by-construction techniques that ensure that potentially incorrect programs are never even considered; at the other extreme are generate-and-check approaches that completely decouple the search from the verification problem. In between, though there is a rich space of techniques where the correctness constraints drive the search and steer the synthesizer towards solutions that are actually easy to verify.

17.1 Framing the synthesis problem

Defining correctness

Attempt 1: Not enough

Precondition: $n \geq 0 \wedge n \leq len(in)$

out=Sort(n, in)

Postcondition: $\forall i.\, 0 \leq i < n-1 \Rightarrow out[i] \leq out[i+1]$

$len(out) = n$

Defining correctness

Attempt 3: Still not enough

Precondition: $n \geq 0 \wedge n \leq len(in)$

out=Sort(n, in)

Postcondition: $\forall i.\, 0 \leq i < n-1 \Rightarrow out[i] \leq out[i+1]$

$len(out) = n$

$\forall i.\, 0 \leq i < n-1 \Rightarrow \exists j\ in[j] = out[i]$

$\forall i.\, 0 \leq i < n-1 \Rightarrow \exists j\ in[i] = out[j]$

Defining correctness

Attempt 4: Is it enough?

Precondition: $n \geq 0 \wedge n \leq len(in)$

out=Sort(n, in)

Postcondition: $\forall i.\, 0 \leq i < n-1 \Rightarrow out[i] \leq out[i+1]$

$len(out) = n$

~~$\forall i.\, 0 \leq i < n-1 \Rightarrow \exists j\ in[j] = out[i]$~~

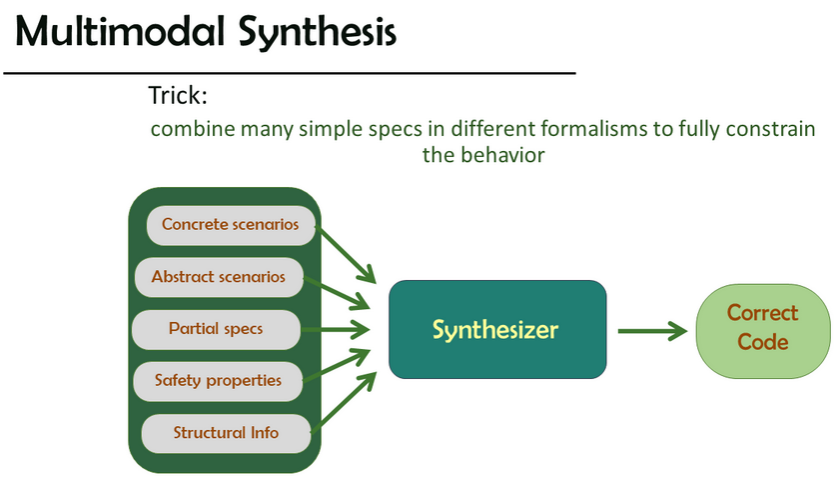
$\forall i.\, 0 \leq i < n-1 \Rightarrow \exists j\ in[i] = out[j]$

$\exists p: int \to int.\, \forall i.\, i < n \Rightarrow\ out[i] = in[p(i)] \wedge \exists j \in [0, n).\, p(j) = i$

The most traditional form of specification in this space is through the use of pre-conditions and post-conditions. A pre-condition is a predicate that all valid inputs to a function must satisfy, while a post-condition is a predicate that all outputs must satisfy. Informally speaking, the precondition is a promise that the environment makes to the function about the parameters it will pass

and the context in which the function will be invoked. The post-condition is a promise that the function makes, conditional on the environment having lived up to its promise.

One of the challenges in attempting full functional specification of behavior is that it can sometimes be difficult to pin down what the expected behavior of a function ought to be. As an example, illustrated in the figure, consider the problem of fully specifying the behavior of a `sort` function. Attempt 1 seems reasonable, but it is clearly not enough. For example, given the input `[3, 2, 4, 2, 1]`, an output of the form `[5, 5, 5, 5, 5]` would clearly satisfy the specification despite not conforming to our expectation of the behavior of `sort`. Attempt 2 seems more promising, since it also requires that every element on the output matches some element of the input, but it is still insufficient, since it would accept `[3,3,3,3,3]` as a valid output. Attempt 3 further strengthens the definition with the requirement that every element in the input also be present in the output, but even that is still insufficient, as it would allow an output such as `[1,2,3,4,4]`. Attempt 5 is actually fully consistent with our expectation of the behavior of `sort`, but it is also fairly unintuitive. It introduces a helper function p that is required to be a permutation function that fully characterizes which output came from which input.

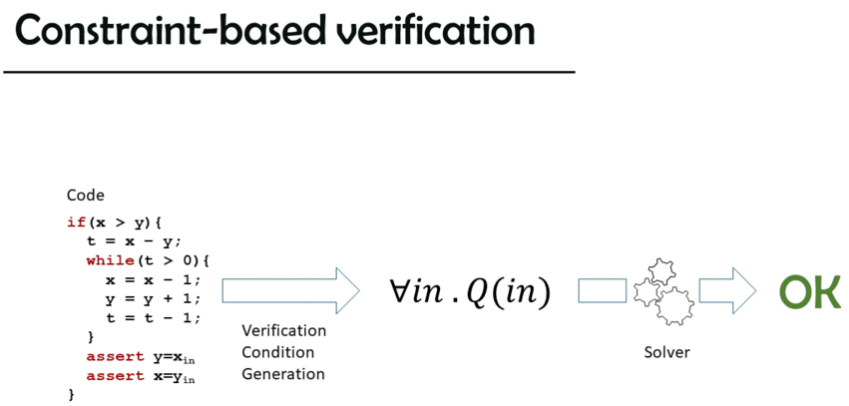


At some level, this problem is unavoidable, at least until the synthesizer is capable of reading our mind. However, it can be significantly ameliorated through *multi-modal interaction*. The idea of multi-modal synthesis was first articulated in the work of Singh and Solar-Lezama on storyboard programming [1], is that for complex programs, it will be difficult to fully articulate the behavior of a program in any one formalism. However, different formalisms can make it easier to articulate different properties, so by allowing the programmer to provide specifications in multiple different formalisms, a user can fully

constrain the behavior of the desired program. For the running example, Attempt 3 may not be enough to fully specify the behavior of the program, but combined with a concrete input/outptu example that exposes the expected behavior when there are multiple instances of the same value may be enough to fully constrain the implementation to the correct one given even a very general template of the desired implementation.

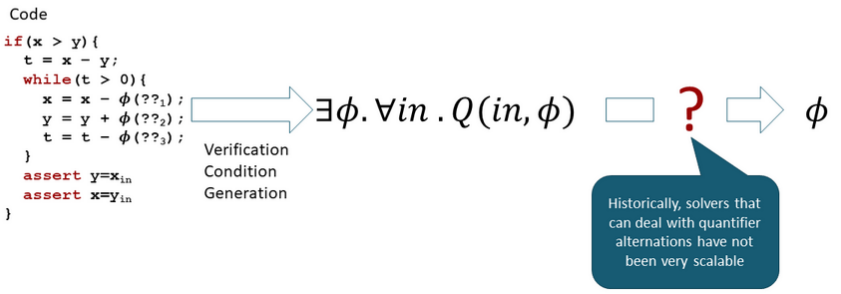
17.2 Interplay of verification and synthesis

At a high-level, there are four main mechanisms that we will be exploring to ensure the correctness for programs: constraint-based techniques based on symbolic execution, abstract interpretation, type-based analysis and deductive verification.



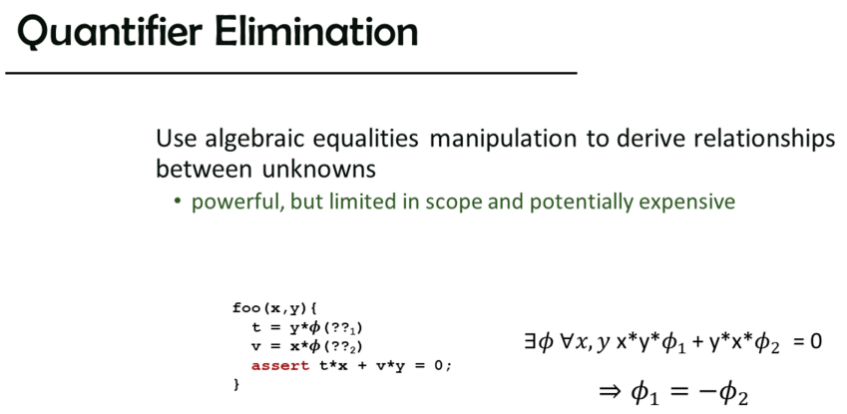
The constraint-based techniques actually constitute a broad category of verification techniques, but the general setup is illustrated in the figure. In this approach, a program is converted into a predicate of the form $\forall in.Q(in)$ that must hold for all inputs (and also potentially for all values of temporary variables introduced by the symbolic execution process). If the predicate indeed holds, then we can conclude that the program is correct with respect to its specification.

The method of generating verification conditions that we will present in Lecture 13, is a sound but incomplete member of this category. Sound means that if it claims correctness (if the universally quantified formula is valid), then we can be sure that the program is correct, but if the formula does not hold (if we can find an in for which Q is not true), then the program may or may not be incorrect, so it is incomplete because it cannot verify all correct programs. Bounded model-checking, which will be presented in Lecture 12, is by contrast an unsound but complete instance of this class of techniques.



These solver-based techniques generalize easily to the synthesis case with the introduction of an extra quantifier over the space of programs. Historically, however, the problem of solving general formulas with quantifier alternation has proven quite difficult, and even though there are off-the-shelf solvers for quantified boolean formulas with arbitrary quantifier alternation, they are not widely used in the synthesis community because they tend not to be very efficient for synthesis problems. Instead, there are two major techniques to deal with this quantifier alternation: quantifier elimination and *Counterexample Guided Inductive Synthesis* (CEGIS).

The main idea in quantifier elimination is to algebraically eliminate any variables that appear in the universal quantifier. The major downside of quantifier elimination is that it can be very expensive. For boolean variables, each variable eliminated can grow the size of the formula exponentially, and once we go beyond linear arithmetic there may not even be effective procedures for variable elimination. However, in some instances it can be very efficient, and even when it cannot be applied fully, it can be a useful preprocessing step before attempting other methods.



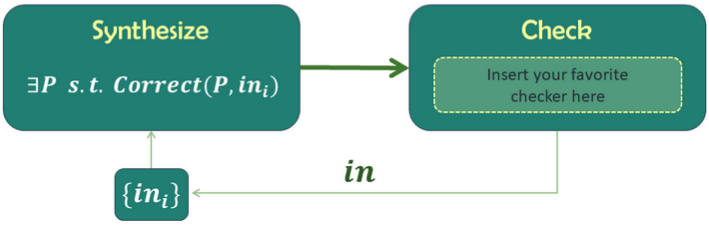
As an example, consider the simple program in the figure. In the example, the correctness condition reduces to a simple predicate, and simple algebraic manipulation completely eliminates the universally quantified variables from the predicate, leaving us with a constraint that all valid assignments ϕ must satisfy.

17.3 Counterexample Guided Inductive Synthesis

The key ideas behind CEGIS were first described by Solar-Lezama, Tancau, Bodik, Seshia and Saraswat in a 2006 paper [2], although it wasn't until a later paper in 2008 [3] that the term CEGIS was coined by Solar-Lezama, Bodik and Jones and the algorithm was explained in the general form used here.

CEGIS is a form of generate and check, where a synthesizer generates candidate programs that are checked by an off-the-shelf checking procedure. The key idea in CEGIS, however, is to use a checker capable of producing counterexample inputs. This allows us to use an inductive synthesis procedure instead of simply producing proposals blindly. The inductive synthesis procedure is forced to produce proposals that work for all the counterexample inputs discovered so far. The idea is to avoid producing new candidates that will fail in similar ways from previously rejected programs.

CEGIS

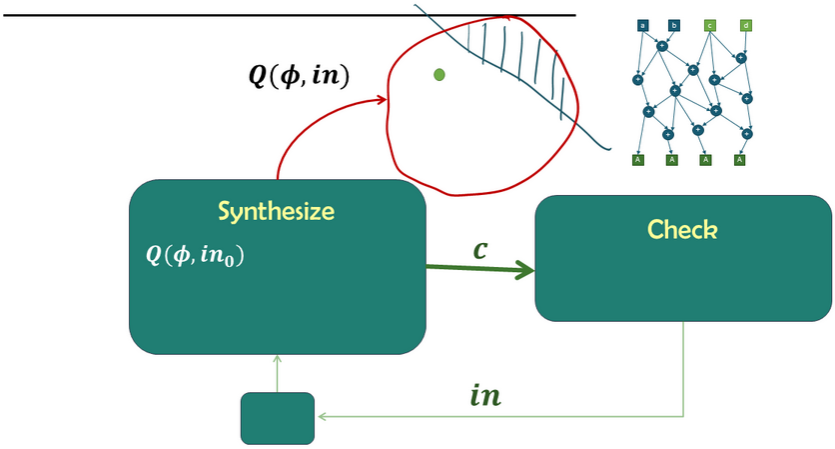


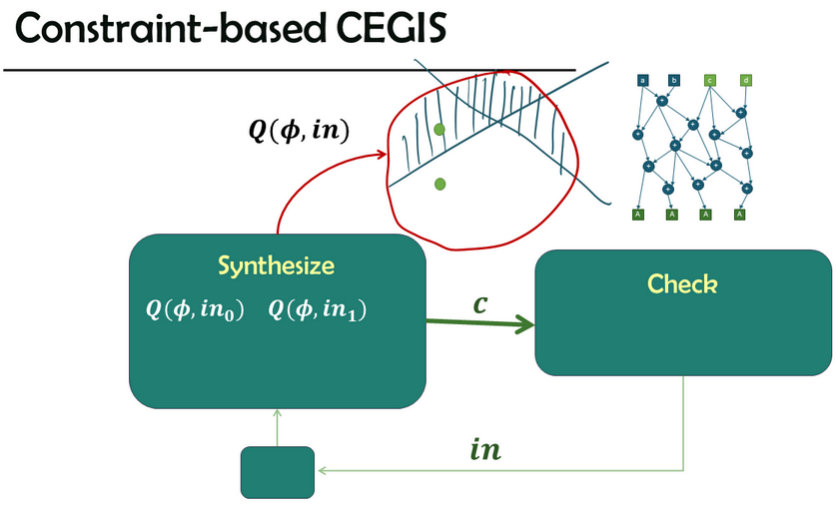
Constraint-based CEGIS

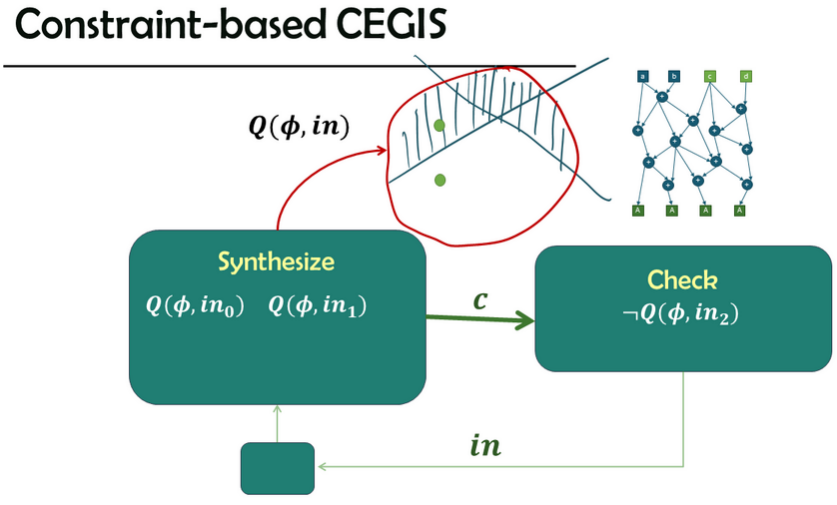
$Q(c, in)$

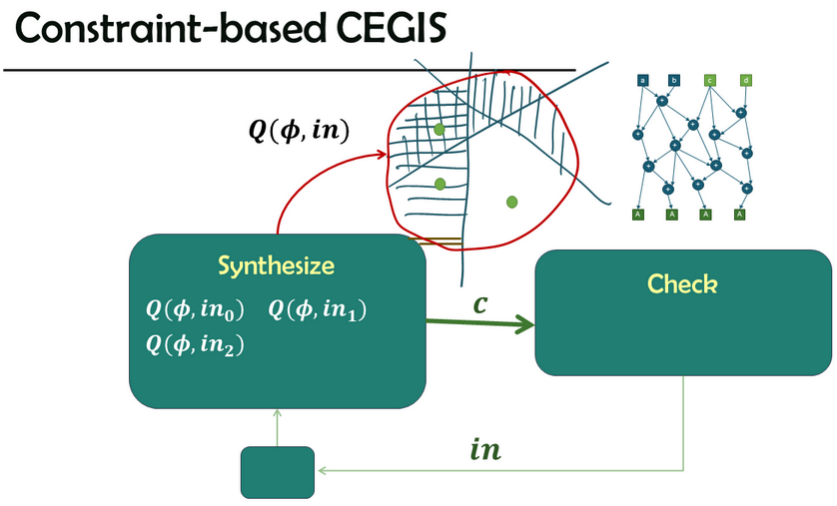


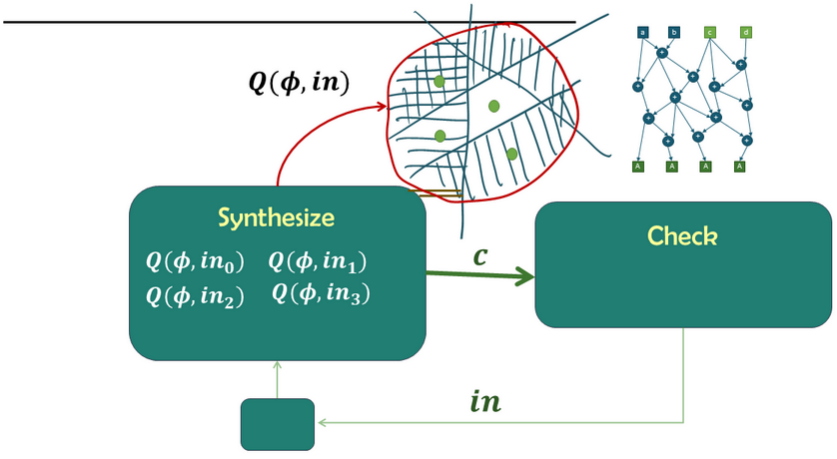
Constraint-based CEGIS

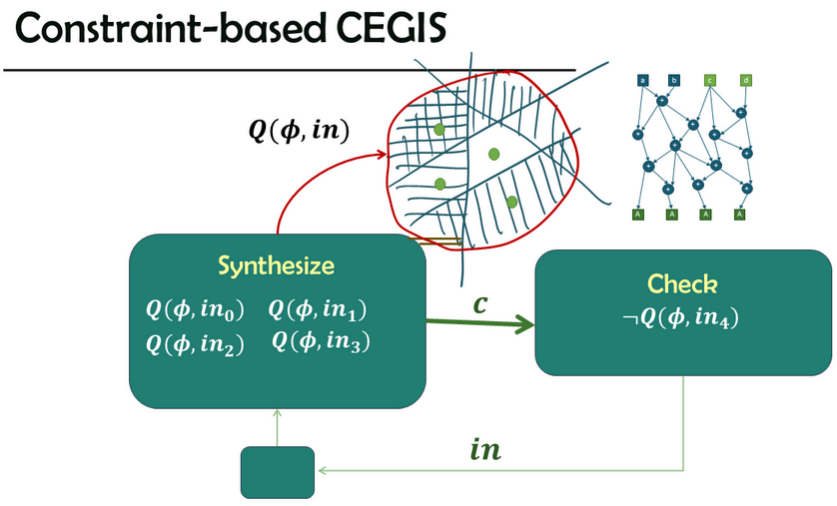


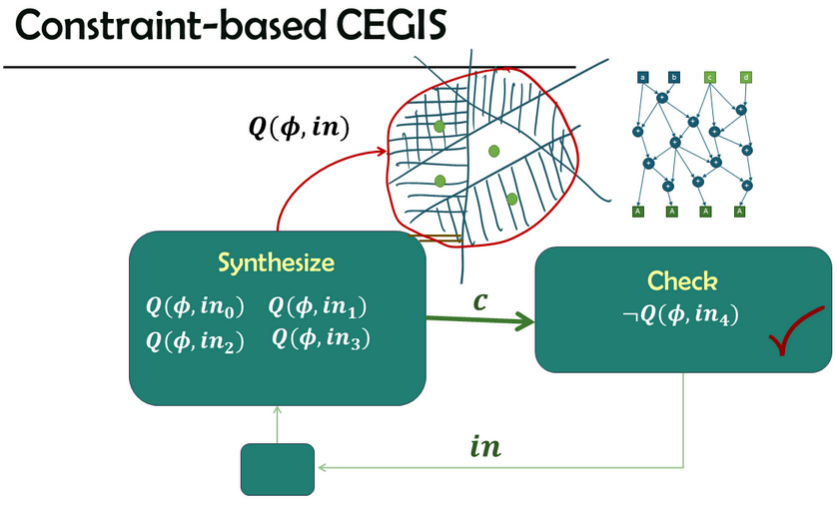

Constraint-based CEGIS
$Q(\phi, in)$
Synthesize
$Q(\phi, in_0)$ $Q(\phi, in_1)$
c
Check
in


Constraint-based CEGIS
$Q(\phi, in)$
Synthesize
$Q(\phi, in_0)$ $Q(\phi, in_1)$
c
Check
$\neg Q(\phi, in_2)$
in


Constraint-based CEGIS
$Q(\phi, in)$
Synthesize
$Q(\phi, in_0)$ $Q(\phi, in_1)$
$Q(\phi, in_2)$
c
Check
in


Constraint-based CEGIS
$Q(\phi, in)$
Synthesize
$Q(\phi, in_0)$ $Q(\phi, in_1)$
$Q(\phi, in_2)$ $Q(\phi, in_3)$
c
Check
in





As the figure illustrates, every time a candidate proposal is rejected by the checker and a counterexample is generated, the counterexample has the effect of not just eliminating the offending program, but it rules out every other program that would have also failed on that same input. In the figure, the red circle represents the set of viable programs. Initially, the synthesizer searches for a program that works for an initial random input in_0. Every subsequent input in_i is the result of solving a checking problem where we look for an input that fails for the current candidate program. The process continues until it is not possible to find additional counterexamples, at which point the process stops.

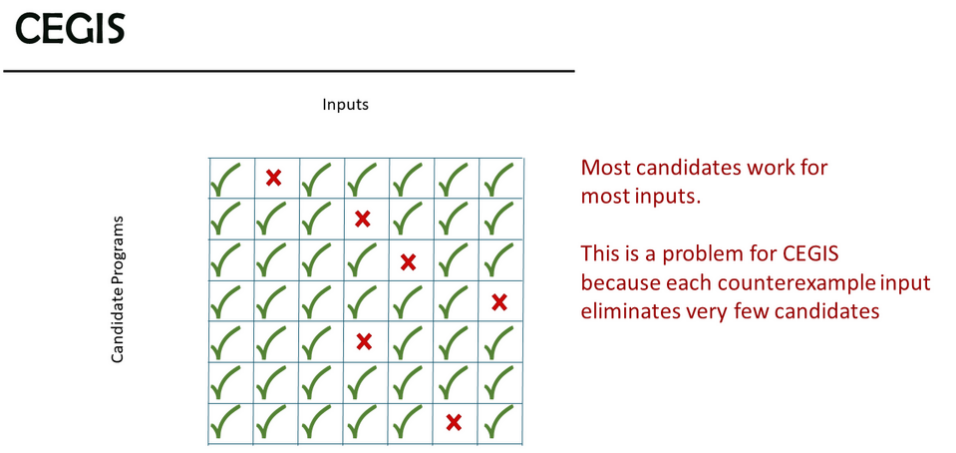
CEGIS was first proposed in the context of constraint-based synthesis, where the inductive synthesis problem itself could be expressed as a series of constraints on the parameters ϕ, but the idea is actually more general. One of

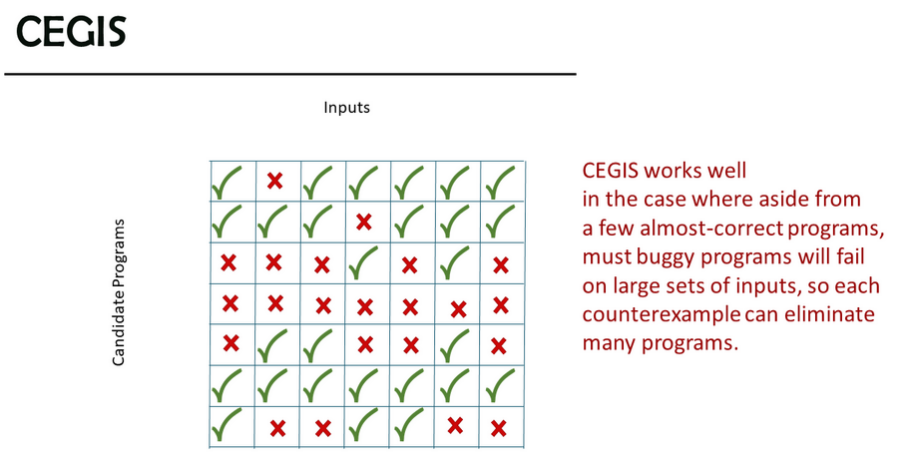
the features that has made CEGIS so popular is the ability to mix and match different inductive synthesis procedures with different checking mechanisms. In that way, CEGIS provides a bridge between all the inductive synthesis techniques studied in the previous unit and the expressive specifications that will be explored in this unit.

17.4 Beyond CEGIS

CEGIS is very powerful, but it is not magic, and there are well documented instances where it fails very dramatically. The simplest program that illustrates the failures of CEGIS is the program below:

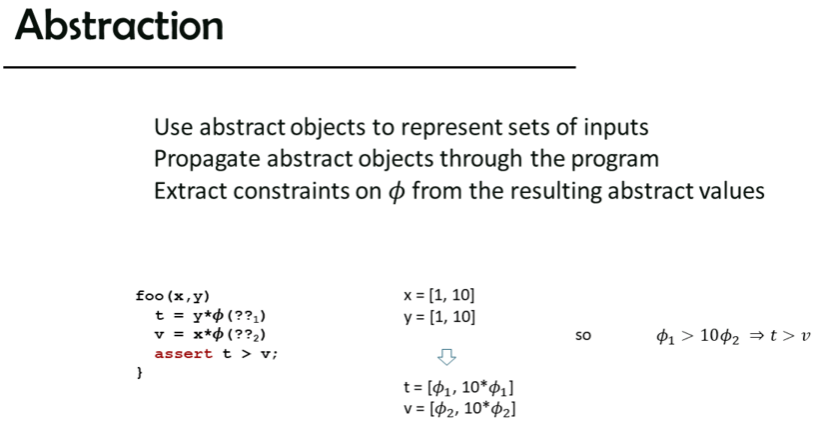
```
void foo(int x, int y){
    assert x != y+??;
}
```





There is clearly no value for the unknown constant that would guarantee that `x != y + ??` for all inputs x and y. However, for any finite set of (x,y) pairs, it is possible to find a constant that will make the inequality hold. To better understand the nature of this example, consider the grid in the figure. As the figure illustrates, CEGIS works best in cases where the buggy program will fail on large sets of inputs. By contrast, the example above has the property that a buggy program will seem to work fine on most inputs, failing only on a single very specific input pair. The biggest sterngth of CEGIS, the fact that it only has to consider small number of inputs at a time, is also its biggest weakness when it comes to these kinds of problems.

In contrast, a number of other synthesis approaches have been proposed that rely on the other kinds of verification mentioned in the beginning: abstract interpretation, type-based analysis and deductive verification. The key feature in these kinds of approaches is a much tighter coupling between the synthesis and verification techniques. The key observation in all of these approaches is that synthesis can in fact be easier than verification, because unlike traditional verification, where the verifyer has to be able to certify whatever crazy program the developer writes, the synthesizer can be steered to produce programs that are easy to verify by the chosen verification technique.



Abstraction. In Lecture 19, we will talk in more detail about Abstract interpretation and abstraction based analysis, but the high-level idea is illustrated by the figure. In this example, we are approximating sets of values with ranges. So for example, the inputs are stated to range from one to ten. It is relatively easy to propagate ranges through a program, even one with unknowns. So in this case, we can see that t ranges from ϕ_1 to $10 * \phi_1$, and that v ranges from ϕ_2 to $10 * \phi_2$. From this information we can infer that as long as $\phi_1 > 10 * \phi_2$, t will be greater than v. This example is quite contrived, but in general the combination of abstraction and synthesis can be very powerful because the synthesizer

can be pushed to choosing values that even a relatively weak analysis is able to prove correct.

Types-based analysis In the context of inductive synthesis, we saw already how type information can help us aggressively prune the space of possible programs. We can take this idea a step further by focusing on even more expressive type systems. Later in Lecture 15, we will explore a more expressive type system based on *refinement types* that will allow us to synthesize programs with non-trivial properties in a modular way.

References

[1] Rishabh Singh and Armando Solar-Lezama. SPT: storyboard programming tool. In *Computer Aided Verification - 24th International Conference, CAV 2012, Berkeley, CA, USA, July 7-13, 2012 Proceedings*, pages 738–743, 2012. doi: 10.1007/978-3-642-31424-7_58. URL https://doi.org/10.1007/978-3-642-31424-7_58.

[2] Armando Solar-Lezama, Liviu Tancau, Rastislav Bodík, Sanjit A. Seshia, and Vijay A. Saraswat. Combinatorial sketching for finite programs. In *Proceedings of the 12th International Conference on Architectural Support for Programming Languages and Operating Systems, ASPLOS 2006, San Jose, CA, USA, October 21-25, 2006*, pages 404–415, 2006. doi: 10.1145/1168857.1168907. URL http://doi.acm.org/10.1145/1168857.1168907.

[3] Armando Solar-Lezama, Christopher Grant Jones, and Rastislav Bodík. Sketching concurrent data structures. In *Proceedings of the ACM SIGPLAN 2008 Conference on Programming Language Design and Implementation, Tucson, AZ, USA, June 7-13, 2008*, pages 136–148, 2008. doi: 10.1145/1375581.1375599. URL http://doi.acm.org/10.1145/1375581.1375599.